

Taller 5

Estimados,

Para este taller deben comparar la ejecución de procesos paralelos en una problemática basada en matrices:

Matriz de costos de viajes utilizando backtracking

Para definir una matriz de costos de distancias de viajes con valores aleatorios en C++, podemos utilizar la librería `<random>` para generar números aleatorios y la clase `vector` para almacenar los valores en una matriz.

A continuación, se muestra un ejemplo de cómo crear una matriz de tamaño n con valores aleatorios entre 1 y 10:

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    int n = 5;
    int matrix[5][5];

    // Inicializa semilla para generar números aleatorios.
    srand((unsigned)time(NULL));

    // Genera e imprime la matriz.
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            // rand() % 10 devuelve un número entre 0 y 9; le sumamos
            // 1 para que sea 1..10
            matrix[i][j] = rand() % 10 + 1;
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Una vez creada la matriz, podemos utilizar backtracking para encontrar el camino más corto desde un punto inicial `start` hasta un punto final `end`. Para ello, podemos crear una función `backtracking` que reciba como parámetros la matriz de costos, el punto actual `current`, el punto final `end`, la distancia recorrida hasta el momento `dist` y una variable para almacenar la distancia mínima `minDist`. La función deberá explorar todos los caminos posibles desde el punto actual hasta el punto final y actualizar la distancia mínima si se encuentra un camino más corto.

A continuación, se muestra un ejemplo de cómo implementar esta función:

```

#include <stdio.h>    // Para printf
#include <stdlib.h>    // Para rand, srand
#include <time.h>      // Para time
#include <limits.h>    // Para INT_MAX
#define N 5

void backtracking(int matrix[N][N], int n, int current, int end, int
    dist, int *minDist, int visited[]) {
    if (current == end) {
        // Si hemos llegado al nodo destino, actualizamos la
        // distancia mínima si es menor que la actual.
        if (dist < *minDist) {
            *minDist = dist;
        }
        return;
    }
    for (int i = 0; i < n; i++) {
        // Si hay arista hacia i y no está visitado
        if (matrix[current][i] != 0 && visited[i] == 0) {
            visited[i] = 1;
            backtracking(matrix, n, i, end, dist +
                matrix[current][i], minDist, visited);
            visited[i] = 0; // backtrack
        }
    }
}

```

```

int main(void) {
    int matrix[N][N];
    int visited[N];
    int i, j;
    int start = 0;
    int end = 4;
    int dist = 0;
    int minDist = INT_MAX;

    srand((unsigned)time(NULL));
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            matrix[i][j] = rand() % 10 + 1;
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    for (i = 0; i < N; i++) {
        visited[i] = 0;
    }
    visited[start] = 1; // Marcamos el nodo inicial como visitado

    // Llamamos a la función de backtracking
    backtracking(matrix, N, start, end, dist, &minDist, visited);

    // Imprimimos la distancia mínima encontrada
    printf("La distancia mínima desde %d hasta %d es: %d\n", start,
        end, minDist);

    return 0;
}

```

En este ejemplo, se ha creado una matriz de tamaño $n=5$ con valores aleatorios entre 1 y 10. Se ha definido el punto inicial `start=0` y el punto final `end=4`. Se ha utilizado una variable

Solicitud

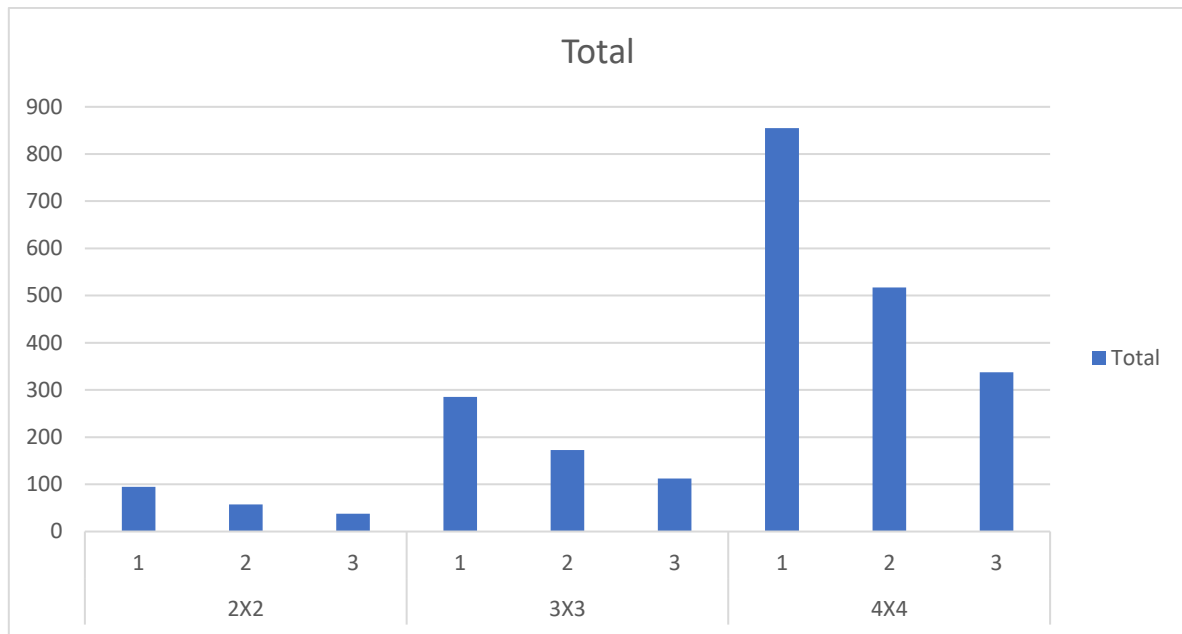
Probar y determinar cuando la ejecución paralela se hace efectivamente más eficiente y estable de acuerdo con la condición de cambio de tamaño de la matriz

Sugerencia: Para encontrar el óptimo, comience probando varios casos con matrices de 2X2, luego progresivamente aumente el tamaño de la matriz midiendo los resultados de tiempos de respuesta de procesamiento, apoyándose en un gráfico para obtener el punto de inflexión en donde se hace más eficiente y estable la mejora de la paralelización.

Referencia de datos recolectados

Matriz	Nucleos	Tiempo
2X2	1	100
2X2	2	60
2X2	3	35
2X2	1	90
2X2	2	55
2X2	3	40
3X3	1	300
3X3	2	180
3X3	3	105
3X3	1	270
3X3	2	165
3X3	3	120
4X4	1	900
4X4	2	540
4X4	3	315
4X4	1	810
4X4	2	495
4X4	3	360
.		
.		
.		
?X?		

Gráfico referencial



Compilación

```
g++ -std=c++0x -o Taller5 Taller5.cpp
```

Nota: En algunos casos el -std puede variar, en donde se puede cambiar a c++11